

request

- error-counter: This counter gets incremented for every 5xx

Counters are separated using the following labels:

- component: the SLI this refers to (for example 'puma' would indicate a request handled by puma)
- environment: 'gstg', 'gprd', ... Only production environments are included in the error budget, but we have them for others.
- featurecategory: The feature this request was for, this is used to later map this to a group based on the info in stages.yml (when imported).
- type: The service this request was for (for example 'api', 'web', or 'git', in case of the puma component).

Performing the calculations

When a person visits the stage group dashboards to see their Error Budget, we perform a calculation using the metrics we hold about how requests have been performing.

Using the formula on the previous page, we use the percentage of successful operations across all 28 days.

When changes are made to the thresholds used in this calculation, it takes 28 days for the effect to be seen because we are summing stored data for the whole period.

---

## SECTION: engineering/frontend/\_index.md

Teams

- Create
- Monitor
- Plan
- Secure
- Verify and Release

Frontend domain experts

You can find engineers with expertise in various frontend domains on the engineering projects page under the following sections:

- GitLab maintainers with domain expertise
- GitLab reviewers with domain expertise

You can reach out to these experts to get help on:

- discussing and defining the architecture of complex frontend features.
- frontend technical topics like Vue, GraphQL, CSS, testing, tooling, etc.

- proposing changes to the cross-domain frontend architecture via an RFC.
- Questions about the frontend for a product area like design management, merge requests, pipelines, etc.

## Frontend group calls

The frontend group has scheduled weekly calls every Tuesday. Since 2021-06-01, these occur at three staggered, time zone friendly times, repeating every three weeks. During these calls, team members are encouraged to share information that may be relevant to share with other members synchronously (Eg. new documentation change, new breaking changes added to master).

To help identify possible conflict areas in a release we encourage members to post a link to a comment in an issue that you're working on that crosses the borders of the domain of your stage group that you feel has the potential of colliding with the work of others.

## Frontend themed call

Each call may include a themed competition at the end, run by the designated DRI.

The purpose of these themes is to add some fun and quirkiness to our team calls (so that we can learn more about each other) but it shouldn't distract or derail into too much off-topic conversation.

## Process

The DRI drives the themed portion of the call, and assigns the winner as the next DRI for that time zone.

1. The DRI decides the theme for the next call for the same time zone, i.e., in three weeks' time.
1. The DRI announces the theme in the frontend Slack channel at least one day in advance.
1. On the next call, at the end of the agenda, the DRI announces the theme and invites participants to share/vocalize their entries.
1. After all entries have been shared, the DRI asks everyone to write which person they liked the answer the most on the meeting's zoom chat without sending.
1. After everyone has written down their vote, the DRI counts down from 3, and on 0 everyone hits enter, sharing their answer at the same time (to prevent voting bias).
1. The entry with the most votes becomes the winner. In the case of a tie, the winners decide which of them becomes the DRI. In case they didn't decide, the DRI can chime in on the decision.
1. The DRI creates an MR to update the table of history below.
1. The DRI replaces their own name with the winner's as the next DRI in the agenda document (internal).
1. goto 1

| Date | Theme | Winner |
|------|-------|--------|
|------|-------|--------|

|            |                                                 |                  |
|------------|-------------------------------------------------|------------------|
| 2018-01-09 | Most interesting hat                            | Jose Ivan Vargas |
| 2018-02-13 | Most interesting shirt/t-shirt (No GitLab swag) | Winnie           |

| 2018-03-06 | Most interesting sunglasses | André |  
| 2018-04-03 | Most peculiar trip souvenir | Eric Eastwood |  
| 2018-05-08 | Grow the best grass or plant (preferably grass) | Lukas |  
| 2018-06-04 | Most interesting sculpture from one piece of paper (maximum size Letter/A4) | Sam |  
| 2018-07-10 | Ornament/Trinket/Thing-that-you-have, that says the most about you | Sam |  
| 2018-08-07 | Most creative Tanuki in any medium | Eric Eastwood |  
| 2018-09-04 | Best animal sketch/drawing | Winnie's wife |  
| 2018-10-02 | Favorite mythical creature of your home country | Tim / Lukas |  
| 2018-11-06 | Convince us that your favorite dish (that you can prepare yourself) is the best! Preferably you share the recipe | Mike |  
| 2018-12-04 | Favorite multi-player game (board game, card game, video game, etc). Describe the rules, how many players, background, and why you like it | Winnie |  
| 2019-01-08 | Favorite children's book | Nathan |  
| 2019-02-05 | Best feature of your house/office/co-working space. If possible, show it! <br /><br /> Some examples:<br /> - The ocean view from your living room window<br /> - Your room exclusively dedicated to Star Wars memorabilia<br /> - The secret passage you built behind a bookcase | André |  
| 2019-03-05 | Favourite music album of all time? | Fatih |  
| 2019-04-02 | Best picture you took (Feel free to post-edit) | André |  
| 2019-05-07 | skipped due to GitLab Contribute | |  
| 2019-06-04 | What's your favourite movie? | Sam |  
| 2019-07-02 | Show us your pets | Sarah |  
| 2019-08-06 | Share your favorite piece of art: painting, photograph, sculpture, installation, etc. | Himanshu |  
| 2019-09-03 | What is your favourite "What is your favourite?" question? What would you ask? How would you answer it yourself? | Sarah |  
| 2019-10-01 | What is your favorite thing to complain about? | Lukas |  
| 2019-11-12 | What is the best dish you have ever eaten at a restaurant? | Dennis |  
| 2019-12-10 | What is your favorite thing about your hometown? It can be a little-known secret, awesome restaurant, famous story | André, Eipi, Coung and Jiaan |  
| 2020-01-07 | If you could time travel, when would you go and why? | Sam |  
| 2020-02-04 | What (un)usual hobby do you have? | Andrew |  
| 2020-03-03 | What is your favourite food to make? | Chris B |  
| 2020-04-06 | What do you listen to while you are working? | Coung |  
| 2020-05-05 | What skill would you like to master? | Andrew |  
| 2020-06-02 | Describe your perfect date | Sam |  
| 2020-07-07 | Which historical figure would you most like to go for a drink with? | ? |  
| 2020-08-04 | Favorite Pizza Toppings (Wrong Answers Only)| Angelo |  
| 2020-09-01 | Movie / TV Series line you would use in a Merge Request | ? |  
| 2020-10-20 | Spooktacular (Special Halloween edition) | Alexander Turinske |  
| 2021-03-02 | What's your favourite childhood movie? Or the one you've watched the most times? | Frédéric C. |  
| 2021-04-06 | If you could have a super power, what would it be? (No matter what anyone says, being rich is not a superpower, sorry Batman)? | Lena H-K. |  
| 2021-05-11 | If you were not a software engineer, what would you do? | Lukas Eipert |  
| 2021-06-01 | What was the worst haircut you ever had? | Sheldon |  
| 2021-06-22 | What's the piece of tech you miss the most? | Paul S |  
| 2021-08-17 | A kilobyte of fun | Mark |  
| 2021-10-05 | What's the most mind-blowing fact you've ever learned? | Miguel |

| 2022-07-26 | What's the best swag (from any company) you've ever gotten or seen, and why? | Kushal |

| 2022-08-16 | If you got a chance to watch (re-watch) a movie in theater (a movie that released before year 2000), which one would you pick? | No winners, we are all made equal and deserve love |

## Frontend calendar

The frontend Google Calendar was created as a means of unifying and including team members on frontend related discussions.

Frontend meetings should be listed on the calendar unless there is a good reason not to.

Frontend engineers have the calendar permissions to create events on this calendar while GitLab team-members have the permissions to add the frontend calendar to their calendar.

> Note: Due to a bug on Google Calendar, team members are unable to find the calendar by searching.

Team members are recommended to go to the frontend calendar link on their browser to add this calendar to their calendar. This calendar is currently only available within GitLab and is not publicly accessible.

## RFCs for changes to GitLab Frontend

If you would like to propose a new, or a change to an existing, frontend pattern in our codebase, please create an issue in <https://gitlab.com/gitlab-org/frontend/rfcs> so that we can discuss it.

## Videos

### Frontend Skillsharing Initiative

A good way to learn from the collective wisdom of our Frontend team is by checking out the videos available on the Skillsharing Initiative page.

### Frontend Onboarding

Here are some helpful resources that might help your onboarding.

- GitLab: Frontend architecture at GitLab
- GitLab: From Legacy to Vue.js
- How We Do Vue At GitLab

### Technical Interview Metrics

Learn more about how we measure our technical interview metrics in the interview metrics page

---

## SECTION: engineering/frontend/create.md

<!-- Pending create of stable counterparts shortcode and confirmation pages are needed and how

to fix -->

## Mission

The Create team works on the frontend part of GitLab for the Create stage.

For more details about the vision for this area of the product, see the [Create stage] page.

## Team members

```
<%= directteam(roleregexp: /[,&] Create/, managerrole: 'Frontend Engineering Manager, Create') %>
```

## Stable counterparts

```
<%= stablecounterparts(roleregexp: /[,&] Create/, directmanagerrole: 'Frontend Engineering Manager, Create') %>
```

## Useful links

- Create frontend board - this shows work in the current release
- gcreate in Slack

---

## SECTION: engineering/frontend/interview-metrics.md

Our team is consistently iterating on building a more robust technical interview process so that we can:

- raise the bar using data
- improve interviewer consistency (interviewer names are encrypted in the dashboard)
- reduce unconscious bias by standardizing the way we evaluate each candidate

---

## SECTION: engineering/frontend/manage.md

```
<!-- Pending create of stablecounterparts shortcode and confirmation pages are needed and how to fix -->
```

## Mission

The Manage team works on the frontend part of GitLab for the Manage stage.

For more details about the vision for this area of the product, see the Manage stage page.

## Team members

<%= directteam(roleregexp: /[,&] Manage/, managerrole: 'Frontend Engineering Manager, Manage') %>

Stable counterparts

<%= stablecounterparts(roleregexp: /[,&] Manage/, directmanagerrole: 'Frontend Engineering Manager, Manage') %>

Useful links

- Manage frontend assignees board - this shows the tasks assignees in the current release
- Manage frontend tasks board - this shows the priorities of deliverables
- gmanage in Slack

---

## SECTION: engineering/frontend/monitor.md

<!-- Pending create of stablecounterparts shortcode and confirmation pages are needed and how to fix -->

Mission

The Monitor team works on the frontend part of GitLab for the Monitor stage.

For more details about the vision for this area of the product, see the Monitor stage page.

Team members

<%= directteam(roleregexp: /Frontend/, managerrole: 'Fullstack Engineering Manager, Monitor:Respond') %>

Stable counterparts

<%= stablecounterparts(roleregexp: /(?!:.)Monitor(?:Observability)/, directmanagerrole: 'Fullstack Engineering Manager, Monitor:Respond') %>

Useful links

- Monitor frontend board - this shows work in the current release
- gmonitor in Slack

---

## SECTION: engineering/frontend/skillsharing.md

Here we document our efforts of coordinating and facilitating the production of content to improve the sharing of common skills amongst the Frontend group maybe even other groups.

Note: These efforts should not be a replacement of technical documentation but an addition.

How to help?

There are two main ways you can help.

### 1. Creating issues / requests

Create issues on the project of things you'd either like to share with others or things you'd like others to share with you.

This is the best way to document that you're working on something or that you'd like to see particular topics covered by your colleagues.

### 2. Recording and uploading videos

Check the issues and if you find something you're capable of covering, feel free to go ahead and record yourself explaining it.

Remember to assign yourself to the issue you're working on to avoid duplicated efforts.

Format

- Length: 5-10min (shorter ok but not longer)
- Where: GitLab Unfiltered (Add to playlist: Frontend Skillsharing)
- How: Following the No Extra Work rule, record off a Zoom call and upload. Feel free to ask a colleague to join and help you rehearse before recording.

Videos

```
<iframe width="900" height="500"
src="https://www.youtube.com/embed/videoseries?list=PL05JrBw4t0Kpsi2Oc9sd3zrcHfrGj1xW"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
```

---

```
## SECTION: engineering/frontend/verify-release.md
```

```
<!-- Pending create of stablecounterparts shortcode and confirmation pages are needed and how
to fix -->
```

Mission

The Verify and Release teams (formerly CI/CD) work on the frontend part of GitLab for the Verify and Release stages.

For more details about the vision for this area of the product, see the Verify and Release pages.

Team members

<%= directteam(roleregexp: /Release|Verify/, managerrole: 'Frontend Engineering Manager, Release (CD), Verify (CI) & Package') %>

#### Stable counterparts

<%= stablecounterparts(roleregexp: /Release|Verify/, directmanagerrole: 'Frontend Engineering Manager, Release (CD), Verify (CI) & Package') %>

#### Useful links

- Verify and Release frontend board - this shows work in the current release based on assignees
- gverify in Slack
- grelease in Slack
- gverifyrelease-fe in Slack

---

## SECTION: engineering/gitlab-com/\_index.md

The handbook pages nested under "policies" directory are controlled documents, and follow a specific set of requirements to satisfy various regulatory obligations.

Avoid nesting non-controlled documentation at this location.

---

## SECTION: engineering/gitlab-com/policies/backup/\_index.md

#### Purpose

This policy outlines how GitLab performs, monitors, and validates backups and restorations of GitLab.com.

These procedures are critical for ensuring data recovery and disaster recovery for customer data.

#### Scope

GitLab.com's backup strategy includes both monitoring and restore validation.

Customer data is stored in the following locations:

1. All PostgreSQL databases for GitLab.com
1. Object storage for GitLab.com, including packages, LFS, uploads, and CI data
1. The CustomersDot database, which manages subscriptions and purchases
1. Git repositories

#### Not in Scope

1. Customer Data stored in the Redis cache
  1. Data queued for processing



## 1. Sessions and other cached data

### Roles & Responsibilities

| Role                      | Responsibility                                               |
|---------------------------|--------------------------------------------------------------|
| GitLab Team Members       | Ensure adherence to the requirements outlined in this policy |
| Engineering (Code Owners) | Approve significant changes and exceptions to this policy    |

### Procedure

GitLab defines:

- The services requiring backups
- The frequency of backups, data retention periods, and restoration processes
- The procedures for data restoration for Disaster Recovery scenarios

### Backup and Restore

#### PostgreSQL Databases

GitLab.com database backups are taken every 24 hours, with incremental updates every 60 seconds. The data is securely streamed to GCS, encrypted, and retained for 90 days. The CustomersDot database is backed up daily with a retention period of 7 days.

All databases are continuously monitored to ensure successful backups, with alerts triggered for missing backups. Restore processes are validated through regular restoration from disk snapshots and replaying WAL segments.

#### Git Repositories

Repositories are backed up hourly using block-level disk snapshots. These snapshots are stored in multi-region object storage and retained for 14 days. All disks are monitored, with alerts triggered for missing snapshots.

Restore validation is conducted by randomly sampling disks and restoring recent snapshots.

#### Object Storage

Data stored in Object Storage (GCS) benefits from Google's 99.999999999% annual durability and multi-region bucket redundancy. To enhance data protection, Object Versioning and Soft Delete are enabled.

Automated restore validation is not required for Object Storage due to its inherent protections through versioning and soft delete.

## Disaster Recovery

For disaster recovery, backups are validated through periodic restoration exercises called "Game Days" to ensure compliance with recovery time objective (RTO) and recovery point objective (RPO) targets.

GitLab.com is deployed in the us-east1 region across multiple GCP availability zones.

For short-term outages affecting a single zone within us-east1, unaffected zones will scale to restore service.

For the Gitaly service, recovery from backups will be necessary if data loss occurs.

Disaster recovery operations adhere to the Disaster Recovery runbooks.

These procedures target specific services to allow parallelized recovery efforts.

Mock disaster recovery (DR) events are conducted quarterly to simulate incidents affecting one or more services.

These exercises validate DR processes and readiness for real incidents.

During these Game Days, RTO and RPO targets are validated by recording measurements for each procedure.

## Exceptions

Exceptions to this policy will be managed in accordance with the Information Security Policy Exception Management Process.

## References

- Records Retention & Disposal
- Disaster Recovery runbooks
- GameDays

---

## SECTION: engineering/gitlab-com/policies/monitoring/\_index.md

## Purpose

This policy specifies how GitLab ensures continuous monitoring of GitLab.com services, how are SLAs, logging and capacity planning defined.

GitLab implements these measures to ensure compliance with various policies and agreements within the cloud environment, and to ensure that any potential security issues are quickly identified.

## Scope

This policy applies to all GitLab.com monitoring services in support of the statutory, regulatory and contractual requirements of GitLab.

## Roles & Responsibilities

| Role                      | Responsibility                                                                 |
|---------------------------|--------------------------------------------------------------------------------|
| -----                     | -----                                                                          |
| GitLab Team Members       | Responsible for following the requirements in this procedure                   |
| Engineering, Security     | Responsible for implementing and executing this procedure                      |
| Engineering (Code Owners) | Responsible for approving significant changes and exceptions to this procedure |

## Procedure

- GitLab defines how service availability is calculated
- GitLab defines how logs are collected, and analyzed
- GitLab defines capacity management procedures

## Service Availability Definition

Service Availability is the percentage of time during which the platform is in an available state. Other states are degraded and outage.

Each of the user facing services have two Service Level Indicators (SLI): the Apdex score, and the Error rate.

The Apdex score is generally a measure of the service performance (latency).

The Error rate measures the percentage of requests that fail due to an error (usually, a 5XX status code).

A service is considered available when:

1. The Apdex score of the service is above its Service Level Objective (SLO),
1. AND The error rate is below its Service Level Objective (SLO).

An example of available web service; within a 5 minute period:

- At least 90% of requests have a latency within their "satisfactory" threshold
- AND, less than 0.5% of requests return a 5XX error status response.

A service is unavailable, if, for 5 minutes:

- Less than 90% of requests have a latency within their "satisfactory" threshold
- OR, more than 0.5% of requests return a 5XX error status response.

In other words, a service needs to simultaneously meet both of its Service Level Objectives (SLO) targets in order to be considered available. If either target is not met, the service is considered unavailable.

The availability score for a service is then calculated as the percentage of time that it is available. The Availability score for each service combined define the platform Service Availability. This number indicates the availability of GitLab.com for a select period of time.

For example, if service becomes unavailable for a 10 minute period, the availability score will be:

- 99.90% for the week (10 070 minutes of availability out of 10 080 minutes in a week)
- 99.97% for the month (43 190 minutes of availability out of 43 200 minutes in the month)

Finally, the availability metric for GitLab.com is calculated as a weighted average availability over the following services (weights in brackets):

- 1. web (5)
- 1. api (5)
- 1. git (5)
- 1. registry (1)
- 1. ci runners (0)
- 1. pages (0)
- 1. sidekiq (0)

The availability score can be seen on the SLA dashboard, and the Service Availability target is set as an Infrastructure key performance indicator.

## Log management

GitLab.com services emit network, system, and application logs, which are then stored, processed, and searched to provide a basis for incident and security incident investigations.

Logs are stored in a short-term and long-term storage, with their own respective retention policies:

- short-term storage: 7 days
- long-term storage: 365 days

Logs in short-term storage are used to actively monitor application activity, spam events, transient errors, system and network authentication events, security events, and similar.

Logs in long-term storage are used to comply to the Records Retention & Disposal policy. Logs in long-term storage are less granular and have less detail than those in short-term storage.

Detailed overview of architecture, tooling and workflows are listed on the Logging page.

## Capacity Planning

In order to scale GitLab.com infrastructure at the right time and to prevent incidents, GitLab employs a capacity planning process.

The focus of the capacity planning process is to perform historical data analysis to predict future growth.

From this analysis, GitLab leverages a process that provides information around saturation points, which are then delivered to service owners.

Service owners then use this process to act on this information.

The data sources used to feed the forecasting tool are historical saturation and utilization data used as part of standard monitoring of GitLab.com.

The output of our capacity planning process is considered ORANGE, and must be treated per The Data Classification Standard.

Detailed overview of architecture, tooling and workflows are listed on the Capacity Planning page.

## Exceptions

Changes and exceptions to this policy must be approved by the appropriate Infrastructure team responsible for monitoring GitLab.com.

## References

- Records Retention & Disposal

---

## SECTION: engineering/gitlab-com/policies/teleport/\_index.md

## Purpose

To ensure an audited access to our terminal/CLI tools like Database Access and Rails Console, GitLab uses Teleport.

## Scope

The Teleport Access policy applies to all systems within our production environment that require a terminal or CLI access.

## Roles & Responsibilities

| Role | Responsibility |

| --- | --- |

| GitLab Team Members | Responsible for following the requirements in this policy |

| System Owners | Alignment to this policy |

| Code Owners | Responsible for approving changes and exceptions to this policy |

## Procedure

- Teleport access is managed through Okta and is provided as part of a role's baseline group assignment or through an access request with appropriate approval
- Access reviews are performed on a quarterly basis to ensure that all users are appropriate and have appropriate access levels.
- Teleport Audit Logs must be retained for a defined period of 1 year
- Teleport Audit Logs must not be modified and or deleted before the defined time of 1 year
- Access to Teleport Audit log data must be limited based on the principle of least privilege

## Exceptions

Exceptions to this policy will be tracked as per the Information Security Policy Exception Management Process

## References

- What is considered production
- Production Architecture
- Runbook: How to connect to a Database console using Teleport
- Runbook: How to connect to a Rails Console using Teleport
- Teleport

---

## SECTION: engineering/ic-leadership/\_index.md

## Engineering IC Leadership at GitLab: going beyond Senior level

At GitLab, it is expected that everyone is a manager of one. For Individual Contributors (IC) a new type of challenge begins with the Staff Engineer role. Engineering IC Leadership is an alternative career path to Engineering Management.

Just like moving into management, also moving from Senior to Staff changes the day-to-day work and expectations placed on ICs.

Engineering IC Leaders exert technical leverage in their scope of influence.

Like any other leadership role, the focus should be on helping others to improve.

Their impact multiplies with every person they help grow, and the company gets more value when they're not investing time in doing things themselves.

Technical skills developed in their career up until now are still very important, and the role is still hands-on technical. Their technical skills are vast and are developing at a lower rate of change now, but they also get new skills that will drive their career from now on:

Communication, Collaboration, and Leadership.

## What GitLab Engineers Have to Say About IC Leadership

During a Handbook Learning discussion, Eric (former Chief Technology Officer), Engineering IC Leaders, and the Learning and Development team discuss Engineering IC Leadership. We discuss topics during an interactive handbook discussion about what it means to be an IC leader.

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/OXBwtaZ9edQ" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

> Start with a level set. You have an intermediate Engineer, then they become a Senior Engineer, and there's a fork in the road. There is a dual career track where you can choose the "manager track" or the "IC Leadership track." - Eric Johnson (former Chief Technology Officer)

Additional topics covered in the discussion include:

1. What does it mean to be an Engineering IC Leader
2. What are the skills needed to do the job
3. How the role differs from other Engineering management roles
4. Difference between technical credibility and technical leverage
5. Applying iteration to everything you do
6. The four archetypes and how they define the work of IC Leadership

### Technical Leverage

Technical leverage could be described as doing technical work that has a multiplicative impact. It frequently involves activities that are not writing code.

In other words, the impact of the work has a positive effect on more than your personal scope and the immediate or near-term time frame. It should help those around you and in your team, group, or department operate and iterate more efficiently.

Examples of this are:

- Taking part in our architecture process
- Implementing generic solutions to problems that can be easily reused in the future
- Identifying commonalities to multiple problems and solving several at once
- Sharing knowledge via trainings or presentations to boost efficiency
- Identifying and removing roadblocks to the development process

### The four archetypes

Staff Engineers and Engineering Managers shared their perspective on what does Staff level mean at GitLab in an Unfiltered blog post.

Much of what each engineer said overlapped, but each had a unique perspective based on their team and their particular experience within GitLab as an entity.

There are four common archetypes of Staff-plus roles in the industry that could explain this variability their perspective:

- The Tech Lead guides the approach and execution of a particular project. Most frequently they partner closely with a single manager, but sometimes they partner with two or three managers within a focused area. At GitLab, Tech Lead is not only an archetype, but it is also a role
- The Architect is responsible for the direction, quality and approach within a critical area, both today and stretching into the multi-year future horizon. They combine a deep knowledge of technical constraints, user needs, and organization level leadership.
- The Solver digs deep into arbitrarily complex problems and finds an appropriate path forward. Some focus on a given area for long periods, others bounce from hotspot to hotspot as guided by organizational leadership.
- The Right Hand is a partner and an extension of an executive-level manager, borrowing their scope and authority to operate particularly complex organizations. They provide additional leadership bandwidth to leaders of large-scale organizations.

## The four archetypes at GitLab

The four archetypes are patterns of behavior. We expect our Staff+ ICs to exhibit behavior from all the archetypes. The individual inclination will usually make one (or more) of them more prominent, but they all define Engineering IC Leaders.

### Tech Lead

The most common archetype for a new Staff Engineer is the Tech Lead, as a Senior Engineer may start showing Staff level behaviors emerging from their team. At GitLab, this is not only an archetype but also a role assigned to engineers on per-project basis. Read more about this on a dedicated Tech Lead Handbook page.

A Staff Engineer partners with the Engineering Manager and the Product Manager for milestone planning and helps teammates address complexity with their deliverables. This also applies on levels above Staff+, partnering with their peers in Management and Product.

### Architect

At GitLab Architecture is a practice where everyone can contribute but Staff+ Engineers play a fundamental role in that.

> Architecture as a practice is everyone's responsibility, but it is notably ingrained in senior technical leadership roles, where the roles' levels and their sphere of influence determine DRI responsibilities within the practice.

### Solver

Complex problems often require a Staff+ Engineer to handle the first iterations in order to reduce the level of complexity to a manageable state.

Routinely being handed the hardest, least-specified, or most-uncertain work is part of this archetype. As well as guiding other ICs in the team when they're struggling to find a solution.

Other teams may need a Staff+ Engineer on loan. The receiving team may or may not already have a Staff+ Engineer, a Solver deals with the problem at hand, and makes sure the team is empowered to take care of the work once the complexity level is manageable.

### Right Hand

One of the conclusions from our work on Architecture Practice at GitLab is that introducing complex architectural changes can not happen without Staff+ ICs working closely with the decision-makers. This conclusion highlights the need for a close collaboration between Engineering Manager+ and Staff+ Engineers, and it fits very well into the Right Hand archetype definition.

Staff+ Engineers are supposed to broaden the perspectives of their managers. Decision-makers often need the additional context and perspective to make well-informed decisions about investments in the product architecture, understanding expected ROI, and a core technical vision behind such changes.



Building meaningful relationships based on trust will make this whole process smoother and will distribute leadership, both technical and managerial, at every level, from single teams up to department level.

---

## SECTION: engineering/ic-leadership/tech-lead.md

## Tech Lead at GitLab

At GitLab, Tech Lead is an archetype and a role. When we think about "Tech Lead" as an archetype we mean that Staff+ Engineers at GitLab are supposed to exhibit patterns of behavior that make them act as technical leaders, who partner with Engineering Managers and Product Managers to support milestone planning, coordination, sequencing and then help teammates address complexities with their deliverables.

"Tech Lead as an archetype" is an expectation, especially for Staff+ Engineers, but "Tech Lead as a role" can be assigned to any Engineer, regardless of their seniority. The primary factor taken into the account when assigning a Tech Lead role is efficiency, and optionally domain knowledge and/or expertise that might stem from assigning this role to a given individual contributor.

## The Tech Lead Role

At GitLab, a Tech Lead might be an individual contributor with additional responsibilities. Every engineer regardless of their seniority can be a Tech Lead. This is a temporary role that is tied to a specific topic / project. We allow teams to have multiple Tech Leads at the same time for different projects. They may be overseeing the work of additional engineers on the project, and remain aware of the moving pieces across them.

An engineer who is acting as a Tech Lead is not a manager, and the role itself is not an additional seniority level.

## Responsibilities of a Tech Lead

Tech leads wear many hats. Their responsibilities may differ from project to project but may include:

1. Providing technical vision and architecture, defining and evolving the overall technical specification for a given project.
1. Providing technical guidance and mentoring to other engineers on the team.
1. Planning and prioritizing work, organizing the engineering effort by breaking down bigger tasks into smaller actionable items.
1. Tracking progress on commitments and reporting status updates.
1. Risk management, identifying, assessing and managing technical risks that may impact deliverables.
1. Coordination, overseeing the work of others and helping remove blockers.

The Engineering Manager (EM) can also help provide support in project management, but it is up to the tech lead and EM to decide on who owns what to ensure these responsibilities are covered.

## Slack channel

There is Slack channel available  
techleads

(internal) where Tech Leads can collaborate together, and ask questions about the role or an archetype of a Tech Lead. Everyone else is welcome too!

---

## SECTION: engineering/infrastructure/\_index.md

## Mission

The Infrastructure Department enables GitLab (the company) to deliver a single DevOps application, and GitLab SaaS users to focus on generating value for their own businesses by ensuring that we operate an enterprise-grade SaaS platform.

The Infrastructure Department does this by focusing on availability, reliability, performance, and scalability efforts.

These responsibilities have cost efficiency as an additional driving force, reinforced by the properly prioritized dogfooding efforts.

Many other teams also contribute to the success of the SaaS platform. However, it is the responsibility of the Infrastructure Department to drive the ongoing evolution of the SaaS platform, enabled by platform observability data.

## Getting Assistance

If you're a GitLab team member and are looking to alert the Infrastructure teams about an availability issue with GitLab.com, please find quick instructions to report an incident here: [Reporting an Incident](#).

For all other queries, please see the [getting assistance](#) page.

## Vision

The Infrastructure Department operates a fast, secure, and reliable SaaS platform to which (and with which) everyone can contribute.

Integral part of this vision is to:

1. Build a highly performant team of engineers, combining operational and software development experience to influence the best in reliable infrastructure.
1. Work publicly in accordance with our transparency value.
1. Use our own product to prepare, build, deliver work, and support the company strategy.
1. Align our strategy with the industry trends, company direction, and end customer needs.

## Direction

The direction is accomplished by using Objectives and Key Results (OKRs).

Other strategic initiatives to achieve this vision are driven by the needs of enterprise customers looking to adopt GitLab.com. The GitLab.com strategy catalogs top customer requests for the SaaS offering and outlines strategic initiatives across both Infrastructure and Stage Groups needed to address these gaps.

```
{{% include "includes/we-are-also-product-development.md" %}}
```

## Organization structure

(click the boxes for more details)

mermaid

flowchart LR

I[Infrastructure Platforms]

click I "/handbook/engineering/infrastructure/"

I --> DA[Data Access]

click DA "/handbook/engineering/infrastructure-platforms/data-access/"

I --> D[Dedicated]

click D "/handbook/engineering/infrastructure/team/gitlab-dedicated/"

I --> DE[Developer Experience]

click DE "/handbook/engineering/infrastructure-platforms/developer-experience/"

I --> PE[Production Engineering]

click PE "/handbook/engineering/infrastructure-platforms/production-engineering/"

I --> SD[Software Delivery]

click SD "/handbook/engineering/infrastructure/team/delivery/"

I --> TS[Tenant Scale]

click TS "/handbook/engineering/infrastructure-platforms/tenant-scale/"

DA --> GC[Gitaly]

click GC "/handbook/engineering/infrastructure-platforms/data-access/gitaly/"

DA --> Git[Git]

click GG "/handbook/engineering/infrastructure-platforms/data-access/git/"

DA --> DF[Database Framework]

click DF "/handbook/engineering/infrastructure-platforms/data-access/database-framework/"

DA --> DO[Database Operations]

click DO "/handbook/engineering/infrastructure-platforms/data-access/database-operations/"

DA --> DU[Durability]

click DU "/handbook/engineering/infrastructure-platforms/data-access/durability/"

D --> E[Environment Automation]

click E "/handbook/engineering/infrastructure/team/gitlab-dedicated/"

D --> PSS[Public Sector Services]

click PSS "/handbook/engineering/infrastructure/team/gitlab-dedicated/us-public-sector-services/"

D --> Switchboard

click Switchboard ["/handbook/engineering/infrastructure/team/gitlab-dedicated/switchboard/](/handbook/engineering/infrastructure/team/gitlab-dedicated/switchboard/)"

DE --> EA[Development Analytics]

click EA ["/handbook/engineering/infrastructure-platforms/developer-experience/engineering-analytics/](/handbook/engineering/infrastructure-platforms/developer-experience/engineering-analytics/)"

DE --> DT[Developer Tooling]

click DT ["/handbook/engineering/infrastructure-platforms/developer-experience/developer-tooling/](/handbook/engineering/infrastructure-platforms/developer-experience/developer-tooling/)"

DE --> FE[Feature Readiness]

click FE ["/handbook/engineering/infrastructure-platforms/developer-experience/](/handbook/engineering/infrastructure-platforms/developer-experience/)"

DE --> PER[Performance Enablement]

click PER ["/handbook/engineering/infrastructure-platforms/developer-experience/performance-enablement/](/handbook/engineering/infrastructure-platforms/developer-experience/performance-enablement/)"

DE --> TG[Test Governance]

click TG ["/handbook/engineering/infrastructure-platforms/developer-experience/test-governance/](/handbook/engineering/infrastructure-platforms/developer-experience/test-governance/)"

PE --> CC[Cloud Connector]

click CC ["/handbook/engineering/infrastructure/team/cloud-connector/](/handbook/engineering/infrastructure/team/cloud-connector/)"

PE --> Foundations

click Foundations ["/handbook/engineering/infrastructure-platforms/production-engineering/foundations/](/handbook/engineering/infrastructure-platforms/production-engineering/foundations/)"

PE --> Observability

click Observability ["/handbook/engineering/infrastructure/team/scalability/](/handbook/engineering/infrastructure/team/scalability/)"

PE --> Ops

click Ops ["/handbook/engineering/infrastructure/team/ops/](/handbook/engineering/infrastructure/team/ops/)"

PE --> Runway

click Ops ["/handbook/engineering/infrastructure/team/runway/](/handbook/engineering/infrastructure/team/runway/)"

SD --> DB[Build]

click DB ["/handbook/engineering/infrastructure-platforms/gitlab-delivery/distribution/](/handbook/engineering/infrastructure-platforms/gitlab-delivery/distribution/)"

SD --> DD[Deploy]

click DD ["/handbook/engineering/infrastructure-platforms/gitlab-delivery/distribution/](/handbook/engineering/infrastructure-platforms/gitlab-delivery/distribution/)"

SD --> FR[Framework]

click FR ["/handbook/engineering/infrastructure-platforms/gitlab-delivery/framework/](/handbook/engineering/infrastructure-platforms/gitlab-delivery/framework/)"

SD --> RE[Releases]

click RE ["/handbook/engineering/infrastructure-platforms/gitlab-delivery/delivery/](/handbook/engineering/infrastructure-platforms/gitlab-delivery/delivery/)"

SD --> SM[Self-managed]

click SM ["/handbook/engineering/infrastructure-platforms/gitlab-delivery/delivery/](/handbook/engineering/infrastructure-platforms/gitlab-delivery/delivery/)"

TS --> Geo

click Geo ["/handbook/engineering/infrastructure-platforms/tenant-scale/geo/](/handbook/engineering/infrastructure-platforms/tenant-scale/geo/)"

TS --> Organizations

click Organizations ["/handbook/engineering/infrastructure-platforms/tenant-scale/organizations/](/handbook/engineering/infrastructure-platforms/tenant-scale/organizations/)"

TS --> Cells

click Cells ["/handbook/engineering/infrastructure-platforms/tenant-scale/cells-infrastructure/](/handbook/engineering/infrastructure-platforms/tenant-scale/cells-infrastructure/)"

## Technical Roadmap

Infrastructure maintains a Technical Roadmap for planning projects over the short (1y), medium (2y), and long term (3y). This serves as our strategic compass, helping us balance immediate needs with long-term sustainability.

The Technical Roadmap is based on the Product Roadmap, where Product provides the "What" (customer needs) and "Why" (business strategy). Engineers then determine the "How" (technical implementation), while Engineering Managers plan the "When" (scheduling). This comprehensive roadmap emphasizes building high-quality, complete features in a sustainable manner.

The Technical Roadmap serves three key purposes:

1. It helps build engineering excellence by addressing critical areas that might not show up in product backlogs, such as technical debt, performance improvements, platform improvements, and system scalability.

1. It enables the department to be proactive rather than reactive.

By regularly asking key questions like "Where do we see the biggest instability in our systems?" or

"What is generating the most toil?", we can address issues before they become critical problems.

This helps maintain our SLOs and keeps our customers happy.

1. It aligns engineering efforts with business goals, ensuring technical improvements drive GitLab's success.

Each technical roadmap item is prioritized based on business value and strategic alignment.

## Current State

The Infrastructure Roadmap is maintained as a static site. GitLab team-members can review the current technical roadmap, at [infra-roadmap.gitlab.com](https://infra-roadmap.gitlab.com).

### NOTE:

The Infrastructure Roadmap is not publicly available as some of the projects and initiatives may be considered unSAFE.

The site presents the roadmap in a visual manner, showing:

- Dependencies between planned initiatives
- Filtering options by confidence, stage, or tags
- Individual roadmaps for each stage within the department
- Impact analysis through dependency visualization

## Updating the Roadmap

Changes to the Roadmap are made through merge requests to the infra-roadmap project. The data is stored in YAML format, and changes can be made by editing the YAML. This allows for version control and collaborative discussion through the merge request process.

Full instructions for making changes to the Infrastructure Roadmap are available in the project's README.md.

Everyone is encouraged to contribute to the roadmap, whether proposing new initiatives or making smaller changes like updating descriptions or adding links to relevant issues.

## Design

The Infrastructure Library contains documents that outline our thinking about the problems we are solving and represents the current state for any topic, playing a significant role in how we produce technical solutions to meet the challenges we face.

## Dogfooding

The Infrastructure department uses GitLab and GitLab features extensively as the main tool for operating many environments, including GitLab.com.

We follow the same dogfooding process as part of the Engineering function, while keeping the department mission statement as the primary prioritization driver. The prioritization process is aligned to the Engineering function level prioritization process which defines where the priority of dogfooding lies with regards to other technical decisions the Infrastructure department makes.

When we consider building tools to help us operate GitLab.com, we follow the 5x rule to determine whether to build the tool as a feature in GitLab or outside of GitLab. To track Infrastructure's contributions back into the GitLab product, we tag those issues with the appropriate Dogfooding label.

## Handbook use at the Infrastructure department

At GitLab, we have a handbook first policy. It is how we communicate process changes, and how we build up a single source of truth for work that is being delivered every day.

The handbook usage page guide lists a number of general tips. Highlighting the ones that can be encountered most frequently in the Infrastructure department:

1. The wider community can benefit from training materials, architectural diagrams, technical documentation, and how-to documentation. A good place for this detailed information is in the related project documentation. A handbook page can contain a high level overview, and link to more in-depth information placed in the project documentation.
1. Think about the audience consuming the material in the handbook. A detailed run through of a GitLab.com operational runbook in the handbook might provide information that is not applicable to self-managed users, potentially causing confusion. Additionally, the handbook is not a go-to place for operational information, and grouping operational information together in a single place while explaining the general context with links as a reference will increase visibility.

1. Ensure that the handbook pages are easy to consume. Checklists, onboarding, repeatable tasks should be either automated or created in a form of template that can be linked from the handbook.

1. The handbook is the process. The handbook describes our principles, and our epics and issues are our principles put into practice.

## Projects

Classification of the Infrastructure department projects is described on the infrastructure department projects page.

The infrastructure issue tracker is the backlog and a catch-all project for the infrastructure teams and tracks the work our teams are doing unrelated to an ongoing change or incident.

In addition to tracking the backlog, Infrastructure Department projects are captured in our Infrastructure Department Epic as well as in our Quarterly Objectives & Key Results

## Supporting Product Features

We have a model that we use to help us support product features. This model provides details on how we collaborate to ship new features to Production.

## Stable Counterparts

Infrastructure SREs may be aligned with stage groups as stable counterparts.

Stable Counterparts are used as a framework for managing reliable services at GitLab. The framework provides guidelines for collaboration between Stage Groups and Infrastructure Teams.

## Interviewing

The Infrastructure department hires for a number of different technical specialisms and positions across its teams. This Infrastructure Interviewing Guide offers more detail on some of our regular openings, interview process and other useful information related to applying to jobs with us. More information on our current openings can be found on the careers page.

## Common Links

- How we do Incident Management for GitLab.com
- GitLab.com status information

## Slack Channels

- production
- infrastructure-lounge
- incidents-dotcom
- announcements
- feedalerts-general

## General Issue Trackers

- Infrastructure issue queue
- Production incidents, and changes
- Delivery
- Scalability

## Resources

- Production Architecture
- Operational Runbooks
- Environments
- Monitoring
- Readiness Reviews
- Infrastructure Standards

## Other Pages

- On-call Handover
- SRE Onboarding
- GitLab.com data breach notification policy
- Coding at scale

---

## SECTION: engineering/infrastructure/product-management.md

## Responsibilities

The responsibilities of the Infrastructure Product Manager are documented in the job-families page.

## Engagement Model

### Inbound Requests

The Infra PM can help triage and priortize inbound requests to Infrastructure from internal teams and GitLab.com customers.

Types of requests:

1. Dogfooding requests
  - e.g. Runbooks
1. Security and Compliance Requests
1. GitLab.com customer requests in remit of the Infrastructure department:
  - GitLab.com customers, especially enterprises, may often have requests related to operational capabilities or non-functional requirements of GitLab.com (e.g. availability, security, and performance of the service). Requests related to functionality within the application itself should be directed to the appropriate stage team using the standard feature request template.

Examples of requests related to operational capabilities of GitLab.com include:



1. Disaster Recovery SLA
1. Publishing an allocated range of ingress IPs
1. Data locality (e.g. a GitLab.eu instance)
1. Certifications (e.g. FedRAMP)

The Infra PM can help prioritize these requests since they will likely require engineering investment from the Infrastructure department.

How to submit customer requests related to GitLab.com

To submit customer requests related to the operational capabilities of GitLab.com, use the GitLab.com feature request template so that we can gather the necessary opportunity size data to help prioritize this request

### Outbound Requests

The Infra PM can also help drive outbound requests on behalf of Infrastructure to internal teams and GitLab.com customers.

Types of requests:

1. Changes needed within the GitLab application
  - e.g. Kubernetes Migration Blockers
1. Outreach to GitLab.com customers
  - e.g. Document additional egress IP ranges for SaaS
1. Driving cost efficiencies across stages
  - e.g. Registry Cost (internal only)

### General Questions

For general inquiries, use the Infrastructure PM Slack Channel.

### Workflow

To request assistance from the Infrastructure Product Manager, add the ~"infrapm" label to an issue. Next, the Infra PM will consider the issue by moving it to the ~"workflow::problem validation" phase for validation, scoping, and prioritization. After completing these steps, if the Infra PM determines the effort is worth pursuing, they will then follow the established Infrastructure team workflows by adding the appropriate team label and moving the issue to the ~"workflow-infra::Triage" phase for engineering review.

### Prioritization Framework

To prioritize initiatives, the Infra PM uses the framework documented in the Product handbook.

### Issue Board

Infrastructure Product Manager Tasks are tracked on the Infra PM Issue Board

---

## SECTION: engineering/infrastructure/capacity-planning/index.md

## Introduction

In order to scale GitLab infrastructure at the right time and to prevent incidents, we employ a capacity planning process for example for GitLab.com and GitLab Dedicated.

In parts, this process is predictive and gets input from a forecasting tool to predict future needs.

This aims to provide an earlier and less obstrusive warning to infrastructure teams before components reach their individual saturation levels.

The forecasting tool generates capacity warnings which are converted to issues and these issues are raised in various status meetings.

For GitLab.com capacity planning, a report is published and any predicted saturation events result in an issue on the capacity planning issue tracker.

The capacity planning is part of the capacity management policy which can be found on the monitoring policy page.

## Tools

We use and develop Tamland, which is our capacity forecasting tool.

It relies on Facebook's Prophet library for forecasting time series data and generates forecasts on a daily basis.

Tamland's technical documentation can be found in <https://gitlab-com.gitlab.io/gl-infra/tamland>.

GitLab's Capacity Planning strategy is based on the following technologies:

mermaid

graph TD

```
A[Saturation Monitoring Configuration] -->|pushed| B[Prometheus]
B --> C[Mimir]
B --> |alerts for imminent saturation| D[Pager Duty]
C --> |historical saturation data| E[Tamland]
E --> |generates| F[Tamland Report]
F --> G[Capacity Planning Issue Tracker]
```

1. Saturation Monitoring Jsonnet Configuration - for saturation monitoring definition, recording rule generation, short term alerting configuration generation.

1. Prometheus - capturing and processing utilization and saturation metrics over the short-term.

1. Mimir - long-term storage of utilization and saturation metrics.

1. Tamland - the forecasting tool.

1. GitLab CI - running the daily forecasting process.

1. Facebook Prophet - forecasting.

1. GitLab Pages Tamland Report Site - static site hosting of the generated forecasts.
1. GitLab Capacity Planning Issue Tracker - GitLab project used for tracking capacity planning forecast warnings. Issues are created by Tamland directly.
1. GitLab Slack Integration - Slack notifications of new issues in the capacity planning project, to the infracapacity-planning channel.

## Source Data

The forecasting model uses the same saturation and utilization data model that we use to monitor GitLab.com over the short-term. This ensures that anything that we feel is worth monitoring as a potential saturation point will automatically be included in the forecasting model.

Because of this, all services used on GitLab.com are automatically included in the model.

The short-term saturation metric model used on GitLab.com models each resource as a percentage, from 0% to 100%, where 100% is completely saturated. Each resource has an alerting threshold (SLO). If this threshold is breached, alerts will fire and the engineer-on-call will be paged.

The thresholds are decided on a case-by-case basis and vary between resources. Some are near 100% while others are much lower, depending on the nature of the resource, the failure modes on saturation of the resource and the required time-to-mediation. Resources are classed as being either horizontally scalable or not. Horizontally scalable resources are generally considered lower priorities from a capacity planning point-of-view, whereas non-horizontally scalable resources (such as CPU on the primary PostgreSQL instance, for example) require much longer-term strategies for remediation and are therefore considered higher priorities in the capacity planning process.

## Forecasting with Tamland

Tamland relies on Facebook Prophet for generating a forecasting model. Prophet performs analysis of daily, weekly, monthly and yearly trends to forecast a future trend in the data.

Even the most skilled engineer would struggle to predict future saturation, so it's unlikely that a model could do it either. We do not expect it to be totally accurate. Instead, with hundreds of resources on GitLab.com that could potentially become saturated, Tamland's forecasts are a bellweather for changes in trends, particularly upward changes, drawing the attention of engineers who review the data to specific issues.

Tamland will attempt to predict a range of outcomes. For saturation, we focus on the median prediction (50th percentile) and only the upper 80th percentile prediction. The lower 80th percentile is not as important for saturation purposes.

The forecast process, Tamland, runs as a GitLab CI job, e.g. in the gitlab-com project for GitLab.com.

This job will run on a schedule defined in the scheduled pipeline (set to execute daily). The process starts by reading the historical short-term saturation metric data from Mimir, using an hourly resolution across the last 1+ years worth of data.

## Data classification

Per the Data Classification Standard, Tamland's output data is considered ORANGE data and must remain confidential. This includes forecast plots and information about SLO violations.

Therefore, the recommended practice when including Tamland data is:

1. On public facing projects, turn on confidentiality on issues or comments
1. Recordings published to GitLab Unfiltered must be labeled private

## GitLab.com Capacity Planning

### Workflow

Stakeholders: Scalability team and Service Owners

Capacity planning is a shared activity and dependent on input from many stakeholders:

1. The Observability team is the owner of the Capacity Planning process overall - the team oversees the entire process, implements technical improvements to improve our forecasting abilities and helps guide teams to act on associated capacity warnings.
2. Each service we monitor is associated with a Service Owner, who is identified as the DRI to act on capacity warnings and provide input in terms of domain knowledge.

### Observability

1. Tamland analyzes metrics data on a daily basis and creates capacity warning issues if it predicts that a resource will exceed its SLO within the forecast horizon.
1. On a weekly basis, an engineer from the team reviews all open issues in the Capacity Planning tracker following the process described on the Observability team page
  1. Assign legitimate forecasts to the respective Service Owner to review and act on it (see below).
  2. Select the most crucial saturation points to report in the GitLab SaaS Availability meeting based on the impact they would have when fully saturated and how difficult the mitigation might be. To indicate issues like this, we apply the ~"SaaS Weekly" label when we do the weekly triage.
  3. Review forecasts with inaccurate model fit or otherwise obscure predictions, and work on improving their quality. Those issues should be labeled with ~capacity-planning::tune model and not get assigned to the Service Owner directly. Since these model tunings highly benefit from domain insight, the Scalability engineer involves Service Owners to get more information.
  4. Leave a feedback comment for quality assessment, see Quality Assessment and User Feedback below.

### Service Owners

A Service Owner is the individual identified as the DRI for capacity planning for an individual service. This information is covered in the service catalog.

The Service Owner ideally has the closest insight into the service, its recent changes and events and is responsible for its availability and performance characteristics overall. Capacity Planning aims to help the Service Owner to be informed about trends in resource usage

and predicted upcoming resource saturation events, which helps to act early and inform prioritization processes.

For capacity planning, the responsibilities of the Service Owner are:

1. Review and act on automatically generated capacity warnings for their respective service,
2. Provide regular status updates on a capacity warning and link any ongoing related work to it,
3. Feed external domain knowledge back into the forecasting model: Forecast quality is dependent on knowing about external factors like special events, changes we made and other service-specific information.

While many forecasts provide a clear and reliable outlook, not all forecasts will be accurate. For example, a sudden upward trend in the resource saturation metric may be caused by a factor that is known to be temporary - for example, a long running migration. The Service Owner is in the best position to know about these external factors and will evaluate based on all information on-hand to determine whether the forecast is accurate and if the issue requires investigation.

The Service Owner will note down their findings on the issue and get the appropriate actions going to remediate and prevent the saturation event. While the Service Owner is the DRI for the capacity warning, the Infradev Process and the SaaS Availability weekly standup assist with the prioritization of these capacity alerts.

The Service Owner can also decide to change the Service Level Objective, the metric definition or any other forecasting parameters that are used to generate capacity warnings. Please see the related documentation for further information. The Observability team is available to assist, but the work should be owned by the DRI and their team.

If the issue does not require investigation, it is important to follow-up and improve the quality of the forecast or the process to improve the signal-to-noise-ratio for capacity planning. This can include feeding external knowledge into the forecasting model or consider changes in automation to prevent getting this capacity warning too early. The Service Owner is expected to get in touch with Observability to consider and work on potential improvements.

At any time, the Observability team can be consulted and is ready to assist with questions around the forecasting or to help figure out the underlying reasons for a capacity warning.

## Due Dates

We use the due date field to track when the next action is due. For example: the date we expect the issue to drop off the report, or the date we need to take another look at the forecast. We do this because we want to use the capacity planning issue board as the single source of truth. The due date is visible on this board and it is easy to see which issues need attention.

The DRI for an issue is responsible for maintaining the due date and adding status information each time the due date is adjusted.

## Workflow Status Labels

Capacity Planning issues are created without a state. After the initial assessment, one of the following labels should be applied.

- 1. capacity-planning::investigate - this alert requires further active assessment before deciding on a course of action
- 1. capacity-planning::monitor - we need to wait for time to pass to gather further data on this issue to make a decision on how to proceed
- 1. capacity-planning::tune model - we determined the issue isn't relevant at this point in time and intend to tune the forecasting model while we continue to monitor the issue
- 1. capacity-planning::in-progress - there is a mitigation in progress for this alert
- 1. capacity-planning::verification - we have completed work on this issue and are verifying the result

### Saturation Labels

Each issue has saturation labels, indicating which thresholds it exceeds and how. An issue can have multiple saturation labels; for instance, any issue with saturation will, by definition, also have the other three.

- 1. saturation - this issue is predicted (by the median line) to reach 100% saturation in the next 90 days.
- 1. violation - this issue is predicted (by the median line) to reach the saturation threshold (which varies by component) in the next 90 days.
- 1. saturation-80%-confidence - this issue is predicted (by the upper end of the 80% confidence interval) to reach 100% saturation in the next 90 days.
- 1. violation-80%-confidence - this issue is predicted (by the upper end of the 80% confidence interval) to reach the saturation threshold (which varies by component) in the next 90 days.

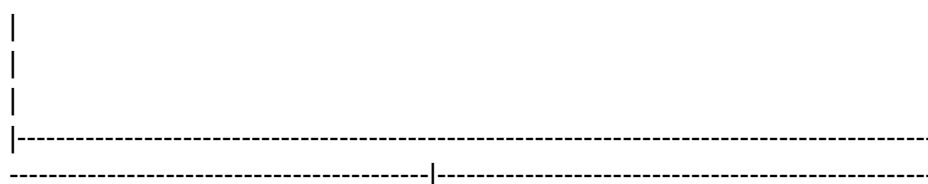
### Other labels

- 1. tamland:keep-open - Used to prevent Tamland from closing the issue automatically. This can be useful to validate the effect of changes we made for a longer period of time until we are confident about the effects.
- 1. Any issue concerning resource saturation or capacity planning in any tracker should have the ~"GitLab.com Resource Saturation" label applied.

### Prioritization Framework

The Scalability:Frameworks team uses capacity planning issues to drive prioritization. By taking saturation data as an input into the planning process, Frameworks team can identity potential projects to balance proactive and reactive work streams.

The prioritization framework uses an Eisenhower Matrix, a 2x2 matrix based on urgency and importance:



-----|  
| Quadrant 1: Do<br>Urgent, Important<br>Reactive: Non-horizontally scalable resources forecasted to saturate 100% in 90 days. | Quadrant 2: Decide<br>Less Urgent, Important<br>Proactive: Non-horizontally scalable resources forecasted to violate hard SLO in 90 days. |  
| Quadrant 3: Delegate<br>Urgent, Less Important<br>Reactive: Horizontally scalable resources forecasted to saturate 100% in 90 days. | Quadrant 4: Deny<br>Less Urgent, Less Important<br>Proactive: Horizontally scalable resources forecasted to violate hard SLO in 90 days. |

Urgent is based on forecast threshold (e.g. 100% saturation vs. hard SLO violation) and important is based on scalable resources (e.g. nonhorizontal vs. horizontal). The following resources are available for prioritization:

Quadrant board  
Issues sorted by priority  
Scoped prioritized labels

### Quality Assessment and User Feedback

In order to determine the quality of forecasts we produce, we rely on and collect feedback from everyone using forecasts and capacity warnings.

This data helps us improve forecasting quality overall and we can also determine which components need further improving or don't qualify for forecasting at all. The following describes what feedback we're looking for and how to give that feedback.

The intended audience is everyone